

INVOICE PROCESSING GEMINI MULTIMODAL PII SAFE — DARBA APRAKSTS

Tika izstrādāta rēķinu apstrādes plūsma, kas apvieno trīs datu avotus (e-pasta metadati, webhook dati, piegādātāju saraksts) ar rēķina attēlu un kas izsauc multimodālo MI modeli (Gemini 2.5 Flash) datu iegūšanai no attēla, un pēc tam Python kods validē izvilktu informāciju pret tiem pašiem datu avotiem.

Projektam ir divas galvenās idejas.

Pirmā — MI tiek izmantots kā viens komponents lielākā sistēmā, nevis kā lēmumu pieņēmējs. Modelis iegūst faktus no attēla. Lēmumus pieņem deterministisks (paredzams un atkārtojams) Python kods.

Gemini uzdevums ir izvilkt faktus no attēla — `invoice_id`, `supplier`, summas, line items. Lēmumu loģika dzīvo atsevišķi `validation.py` failā kā deterministisks Python kods, kas neatkarīgi salīdzina šos faktus pret webhook un piegādātāju datiem. Šī sadalījuma rezultātā katram lēmumam ir pamatots un pārbaudāms.

Otrā — projekts demonstrē PII (Personally Identifiable Information) anonimizācijas principu uz teksta datiem (e-pasti, telefoni). Tas apzināti neaptver divas svarīgas kategorijas:

- personu vārdu un adresu atpazīšanu, kas prasa NER modeli;
- PII maskēšanu pašā rēķina attēlā, kas prasa OCR + lauku maskēšanu. Reālā sistēmā tieši attēlā ir visjutīgākā PII, tāpēc pilnvērtīgs risinājums sāktos ar attēla apstrādi.

Pirms datu nosūtīšanas uz ārējo MI serveri, PII teksta laukos tiek aizvietoti ar marķieriem. Atbilstības tabula tiek glabāta atmiņā un izmantota modeļa atbildes re-identifikācijai. Ārējais MI nekad nesaņem oriģinālos e-pastus, telefona numurus vai citu PII.

Novērojumi

- Telefonu regex paraugs sākotnēji pieņēma datumu 2024-11-15 par telefona numuru, jo datuma formāts atbilst paraugam "skaitļi-atdalītāji-skaitļi". Tas tika atklāts pirmajā anonimizācijas testā — `<PHONE_1>` parādījās vairākās citās vietās. Risinājums — telefona regex tika padarīts precīzāks, prasot vai nu + valsts koda prefiksu, vai amerikāņu stila iekavas ap apgabala kodu;
- Gemini atgriezta valūta nekonsekventi — dažreiz kā USD, citreiz kā \$. Tā bija paredzama MI aģenta uzvedība starp izsaukumiem. Tieši tāpēc validācijai jābūt deterministiskai — ja tā paļautos uz Gemini izvadi tieši, sistēma būtu nestabila. Risinājums — normalizācijas slānis pirms validācijas, kas kartē simbolus uz ISO 4217 kodiem. Tagad sistēma vienmēr atgriež USD neatkarīgi no Gemini izvēles;
- Skaitliska salīdzināšana ar `==` floating-point vērtībām var dot kļūdainus rezultātus. `2844.80 == 2844.7999999...` atgriež False, lai gan realitātē ir tā pati summa. Risinājums — salīdzināšana ar pielaidi ($\text{abs}(a - b) \leq 0.01$) finanšu summām;
- Sākotnējais Gemini 2.0 modelis (google/gemini-2.0-flash-001) tika izņemts no OpenRouter platformas projekta darba laikā. Tas tika atklāts ar 404 kļūdu API izsaukumā. Analīze tika veikta ar atsevišķu pārbaudes skriptu, kas izsauc /models endpoint un izdrukāja visus pieejamos Gemini variantus. Modelis tika nomainīts uz google/gemini-2.5-flash;

- Projekta mapes pārvietošana radīja virtuālās vides problēmas, jo .venv iekšā ir cietkodēti ceļi uz oriģinālo Python instalāciju. Pēc dzēšanas pāri palika sajaukti faili no divām dažādām Python versijām. Risinājums — pilnīga .venv dzēšana ar Remove-Item -Recurse -Force un atjaunošana no requirements.txt. Šī situācija apstiprināja, cik svarīgi ir uzturēt requirements.txt no projekta sākuma — bez tā atjaunošana nebūtu iespējama;

Anonimizācijas slānis

Anonimizācijas modulis (anonymizer.py) ir izveidots kā Python klase Anonymizer, kas uztur iekšējo atbilstības tabulu starp PII vērtībām un marķieriem. Katrai jaunajai PII vērtībai tiek piešķirts unikāls marķieris (<EMAIL_1>, <EMAIL_2>, <PHONE_1>), vērtībām, kas atkārtojas ir tas pats marķieris. Modulis iet cauri sarežģītām datu struktūrām — neatkarīgi no tā, cik dziļi tās ir ligzdotas — un anonimizē visus teksta laukus, atstājot citus tipus (skaitļi, none, boolean) nemainīgus. Galvenā atziņa ir tāda, ka modulis izmanto tikai regex paraugus un ir paredzēts demonstrācijai.

Būtiskākais

Atbildības sadalījums starp MI un kodu. MI ir spēcīgs faktu izvilkšanā no attēla un vājš tur, kur vajadzīga precīza, atkārtojama loģika un izsekojamība. Sadalot atbildību — Gemini iegūst datus, Python pieņem lēmumu — sistēma kļūst auditējama. Katrs lēmums tagad ir reproducējams un izskaidrojams ar konkrētiem skaitļiem. Piemēram, REJECT lēmums tagad nāk ar paziņojumu "image=2844.8 vs expected=4365.68 (diff=1520.88)" — kāds vēlāk var apskatīt rezultāta failu un saprast, kāpēc tika pieņemts šis lēmums, neredzot pašu modeli.

Profesionālas izstrādes vides nozīme. Izveidojot projektu VS Code ar virtuālo vidi, requirements.txt, .env, .gitignore un argparse, projekts kļuva pārvietojams un atjaunojams. Tas izpaudās brīdī, kad mapi vajadzēja pārvietot un virtuālā vide salūza — atjaunošana ar pip install -r requirements.txt aizņēma minūti. Bez šī sagatavošanas darba sākumā tas būtu prasījis daudz vairāk.